

However, we must follow the internet standards and ensure our APIs are secure to make our app secure. Let's see how to develop one. First, let's suppose we need to fetch information from Wikipedia; while many open source APIs are available, we can create one easily using Python. Start your browser and open the Wikipedia site. Watch closely the URL pattern in the address. Now, search for something such as Python. Can you observe the URL change in the address bar? That's the indication that the data was sent from the backend and communicated through the web via an API. Now copy the URL and check out the sample code I have given below.

Here I am sharing a code for a sample Python connection of an API using *request* module. Feel free to modify it with the URL we copied from the address bar and play around with it.

```
import requests
def main():
    # Specify the URL to send the GET request to
    url = 'https://www.example.com'

    try:
        # Send a GET request to the specified URL
        response = requests.get(url)

        # Check if the request was successful (status code
        200)
        if response.status_code == 200:
            # Print the content of the response (the webpage
            HTML)
            print('Response Content:')
            print(response.text)
        else:
            # Print an error message if the request was not
            successful
            print(f'Error: Failed to retrieve content. Status
            code: {response.status_code}')

        except requests.RequestException as e:
            # Handle any exceptions that may occur during the request
            print(f'Error: {e}')

if __name__ == '__main__':
    main()
```

In this example:

- We first import the *Requests* library using *import requests*.
- Inside the *main()* function:
 - We specify the URL of the website we want to send a GET request to (*url = 'https://www.example.com'*).
 - We use *requests.get(url)* to send a GET request to the specified URL and store the response in the *response* variable.

- We check if the request was successful (status code 200) using *response.status_code*.
- If the request was successful, we print the content of the response (the HTML of the webpage) using *response.text*.
- If the request was not successful, we print an error message with the status code.
- We handle any exceptions that may occur during the request using a *try-except* block.

You can replace *https://www.example.com* with the URL of any website you want to make a request to. This example demonstrates a basic usage of the *Requests* library for making HTTP requests in Python.

Here are a few Python modules that allow interaction for web APIs; each module has its use case depending upon the role it plays.

- Requests
- Urllib3
- Aiohttp
- GRequests
- HTTPx
- Uplink

You can easily install these in your system using *pip*.

The following online platforms provide valuable resources for learning and accessing APIs, enhancing our understanding and working capabilities with them. Notably, they offer access to a wide range of free APIs through the open source community.

1. Rapid API (<https://rapidapi.com/>)
2. Programmable Web (<https://www.apistack.io/apps/programmableweb>)
3. GitHub (<https://github.com/>)
4. PostMan (<https://www.postman.com/>)
5. API List (<https://apistack.fun/>)

These platforms offer a wealth of APIs for different purposes, whether building web applications, mobile apps, or integrating services into existing systems. Explore their catalogues and documentation to find APIs that suit your project requirements.

Python's popularity can be attributed to its ease of learning, the vibrant open source community, the boom of AI applications, and its adoption by industry giants and startups alike. Additionally, we have gained insight into developing Python APIs, laying the groundwork for creating seamless interactions between the different components of our software projects. **END** 🐧

By: S.V. Rohith

The author specialises in RPA, Python, and Web 3.0, and has over five years of expertise in creating and managing Telegram bots.